Household Electricity Consumption Forecasting Using Machine Learning

Project type: End-to-end Predictive Analytics / KDD
Dataset: Individual Household Electric Power Consumption — UCI Machine Learning Repository

Objective

Predict household active power consumption **one hour ahead** using historical electricity measurements and time-based features.

Notes

- This notebook downloads the official UCI dataset using `ucimlrepo` if available, with a direct ZIP download fallback.
- The raw data is sampled every minute. We aggregate it to hourly observations and predict the next hour's mean `Global_active_power`.
- A chronological split is used to avoid future-data leakage.
- Metrics are reported on the held-out test period. Results are produced by running the notebook; do not claim unrun scores in the report.

1. Imports and configuration

```
import sys, subprocess, importlib.util, warnings, zipfile, io, urllib.request
warnings.filterwarnings("ignore")

# Install only if a package is missing (uncomment if your environment blocks installs)
required = {"pandas": "pandas", "numpy": "numpy", "matplotlib": "matplotlib", "sklearn": "scikit-learn"}
missing = [pkg for mod, pkg in required.items() if importlib.util.find_spec(mod) is None]
if missing:
    subprocess.check_call([sys.executable, "-m", "pip", "install", *missing])

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor, ExtraTreesRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.inspection import permutation_importance

RANDOM_STATE = 42
DATA_DIR = Path("data")
DATA_DIR.mkdir(exist_ok=True)
print("Python:", sys.version.split()[0])
print("Data directory:", DATA_DIR.resolve())
```

```
Python: 3.13.16
Data directory: /content/data
```

2. Load the UCI dataset

```
# Try the official UCI ZIP first. The file contains semicolon-separated data.
DATA_PATH = DATA_DIR / "household_power_consumption.txt"
ZIP_PATH = DATA_DIR / "household_power_consumption.zip"
UCI_ZIP_URL = "https://archive.ics.uci.edu/static/public/235/individual+household+electric+power+consumption.zip"

if not DATA_PATH.exists():
    try:
        print("Downloading official UCI dataset...")
        urllib.request.urlretrieve(UCI_ZIP_URL, ZIP_PATH)
        with zipfile.ZipFile(ZIP_PATH) as zf:
            candidates = [n for n in zf.namelist() if n.lower().endswith(".txt") and "household" in n.lower()]
            if not candidates:
                candidates = [n for n in zf.namelist() if n.lower().endswith(".txt")]
            if not candidates:
                raise FileNotFoundError("No .txt data file found inside the downloaded archive.")
            with zf.open(candidates[0]) as src, open(DATA_PATH, "wb") as dst:
```

```
                dst.write(src.read())
        print("Dataset saved to:", DATA_PATH)
    except Exception as e:
        print("Automatic download failed:", repr(e))
        print("Download the dataset ZIP from https://archive.ics.uci.edu/dataset/235/individualhouseholdelectricpowerconsur
        print("Extract household_power_consumption.txt into the ./data folder, then rerun this cell.")
        raise

raw = pd.read_csv(
    DATA_PATH,
    sep=";",
    na_values=["?"],
    low_memory=False
)
print("Raw shape:", raw.shape)
display(raw.head())
print("\nColumn types:")
display(raw.dtypes.to_frame("dtype"))
```

```
Downloading official UCI dataset...
Dataset saved to: data/household_power_consumption.txt
Raw shape: (2075259, 9)
```

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2
0	16/12/2006	17:24:00	4.216	0.418	234.84	18.4	0.0	1.0
1	16/12/2006	17:25:00	5.360	0.436	233.63	23.0	0.0	1.0
2	16/12/2006	17:26:00	5.374	0.498	233.29	23.0	0.0	2.0
3	16/12/2006	17:27:00	5.388	0.502	233.74	23.0	0.0	1.0
4	16/12/2006	17:28:00	3.666	0.528	235.68	15.8	0.0	1.0

```
Column types:
```

	dtype
Date	object
Time	object
Global_active_power	float64
Global_reactive_power	float64
Voltage	float64
Global_intensity	float64
Sub_metering_1	float64
Sub_metering_2	float64
Sub_metering_3	float64

3. Data understanding and quality checks

```
print("Rows, columns:", raw.shape)
print("\nMissing values:")
display(raw.isna().sum().sort_values(ascending=False).to_frame("missing_count"))
print("\nDuplicate rows:", int(raw.duplicated().sum()))

# Combine Date and Time into a proper datetime index.
raw["datetime"] = pd.to_datetime(
    raw["Date"].astype(str) + " " + raw["Time"].astype(str),
    format="%d/%m/%Y %H:%M:%S",
    errors="coerce"
)
raw = raw.drop(columns=["Date", "Time"])
raw = raw.dropna(subset=["datetime"]).sort_values("datetime")
raw = raw.drop_duplicates(subset=["datetime"]).set_index("datetime")

numeric_cols = raw.select_dtypes(include=np.number).columns.tolist()
for col in numeric_cols:
    raw[col] = pd.to_numeric(raw[col], errors="coerce")

print("Date range:", raw.index.min(), "to", raw.index.max())
display(raw[numeric_cols].describe().T)
```

```
Rows, columns: (2075259, 9)

Missing values:
```

	missing_count
Global_active_power	25979
Sub_metering_2	25979
Global_reactive_power	25979
Voltage	25979
Global_intensity	25979
Sub_metering_3	25979
Sub_metering_1	25979
Date	0
Time	0

```
Duplicate rows: 0
Date range: 2006-12-16 17:24:00 to 2010-11-26 21:02:00
```

	count	mean	std	min	25%	50%	75%	max
Global_active_power	2049280.0	1.091615	1.057294	0.076	0.308	0.602	1.528	11.122
Global_reactive_power	2049280.0	0.123714	0.112722	0.000	0.048	0.100	0.194	1.390
Voltage	2049280.0	240.839858	3.239987	223.200	238.990	241.010	242.890	254.150
Global_intensity	2049280.0	4.627759	4.444396	0.200	1.400	2.600	6.400	48.400
Sub_metering_1	2049280.0	1.121923	6.153031	0.000	0.000	0.000	0.000	88.000
Sub_metering_2	2049280.0	1.298520	5.822026	0.000	0.000	0.000	1.000	80.000
Sub_metering_3	2049280.0	6.458447	8.437154	0.000	0.000	1.000	17.000	31.000

4. Preprocessing and hourly aggregation

```
# The target is mean active power (kW) in the next hour.
TARGET_SOURCE = "Global_active_power"
if TARGET_SOURCE not in raw.columns:
    raise KeyError(f"Expected column {TARGET_SOURCE!r} not found. Available columns: {raw.columns.tolist()}")

# Convert impossible/invalid measurements to missing rather than silently treating them as real.
# Do not indiscriminately remove outliers: unusual usage may be real.
raw[numeric_cols] = raw[numeric_cols].replace([np.inf, -np.inf], np.nan)

# Resample minute-level readings into hourly mean values.
hourly = raw[numeric_cols].resample("h").mean()
hourly = hourly.rename(columns={TARGET_SOURCE: "target_power_kw"})
hourly = hourly.dropna(subset=["target_power_kw"])

print("Hourly dataset shape:", hourly.shape)
print("Hourly missing values (top 10):")
display(hourly.isna().sum().sort_values(ascending=False).head(10).to_frame("missing_count"))
display(hourly.head())
```

```
Hourly dataset shape: (34168, 7)
Hourly missing values (top 10):
```

	missing_count
target_power_kw	0
Global_reactive_power	0
Voltage	0
Global_intensity	0
Sub_metering_1	0
Sub_metering_2	0
Sub_metering_3	0

datetime	target_power_kw	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_
2006-12-16 17:00:00	4.222889	0.229000	234.643889	18.100000	0.0	0.527778	16.8611
2006-12-16 18:00:00	3.632200	0.080033	234.580167	15.600000	0.0	6.716667	16.8666
2006-12-16 19:00:00	3.400233	0.085233	233.232500	14.503333	0.0	1.433333	16.6833
2006-12-16	3.268567	0.075100	234.071500	13.916667	0.0	0.000000	16.7833

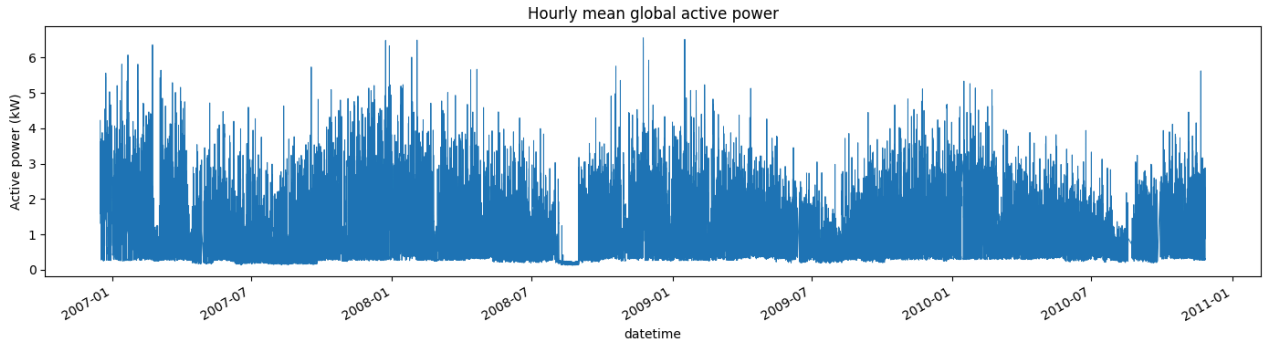
5. Exploratory Data Analysis (EDA)

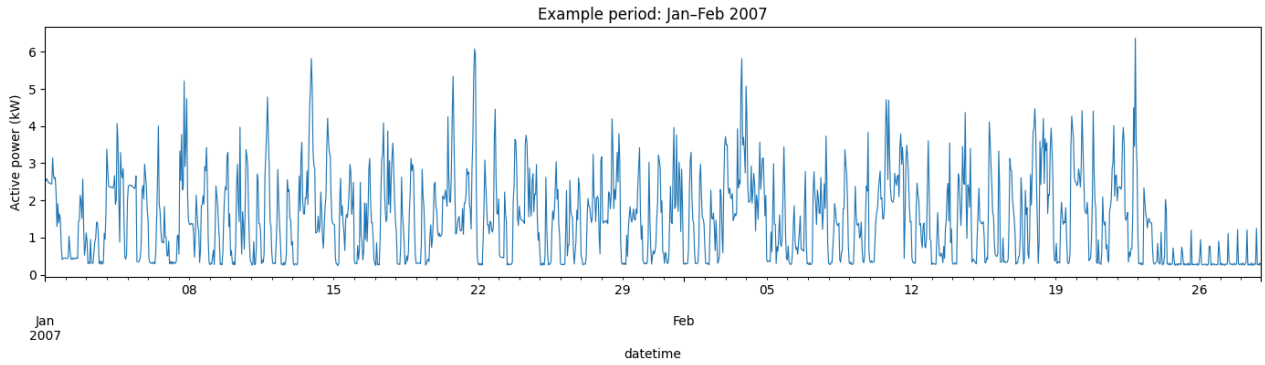
```
fig, axes = plt.subplots(2, 1, figsize=(14, 8))
hourly["target_power_kw"].plot(ax=axes[0], linewidth=0.7, title="Hourly mean global active power")
axes[0].set_ylabel("Active power (kW)")
hourly["target_power_kw"].loc["2007-01":"2007-02"].plot(ax=axes[1], linewidth=0.8, title="Example period: Jan–Feb 2007")
axes[1].set_ylabel("Active power (kW)")
plt.tight_layout()
plt.show()

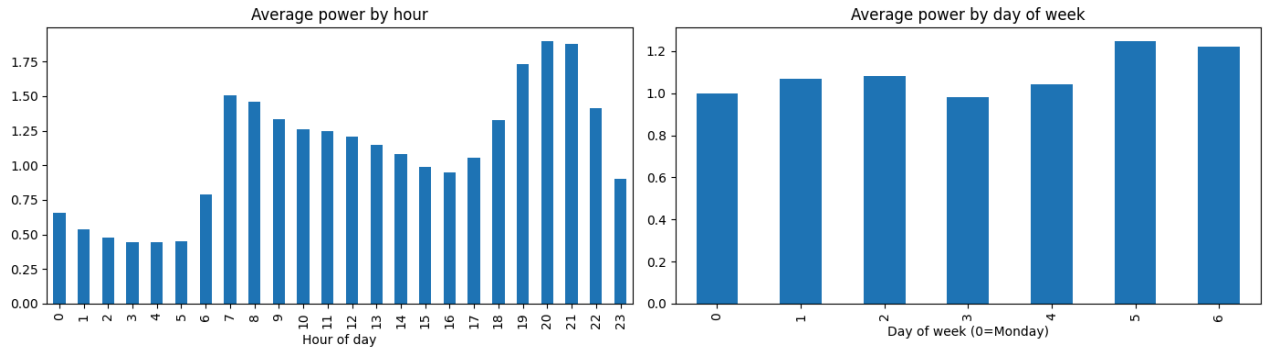
hourly["hour"] = hourly.index.hour
hourly["day_of_week"] = hourly.index.dayofweek
hourly["month"] = hourly.index.month
hourly["is_weekend"] = (hourly.index.dayofweek >= 5).astype(int)
hourly["year"] = hourly.index.year

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
hourly.groupby("hour")["target_power_kw"].mean().plot(kind="bar", ax=axes[0], title="Average power by hour")
hourly.groupby("day_of_week")["target_power_kw"].mean().plot(kind="bar", ax=axes[1], title="Average power by day of week")
axes[0].set_xlabel("Hour of day")
axes[1].set_xlabel("Day of week (0=Monday)")
plt.tight_layout()
plt.show()

print("Hourly target distribution:")
display(hourly["target_power_kw"].describe().to_frame("value"))
```







Hourly target distribution:

	value
count	34168.000000
mean	1.091728
std	0.897619
min	0.124000
25%	0.341925
50%	0.802850
75%	1.579342
max	6.560533

6. Feature engineering and forecasting target

We predict the **next hour's** mean active power. Features at time *t* are used to predict consumption at *t + 1 hour*. Lag features are created from the observed target series and rolling statistics use only past observations. The shift below is important: it makes the prediction horizon explicit.

```
model_df = hourly.copy()

# Calendar features for the hour whose consumption is being forecast.
model_df["hour"] = model_df.index.hour
model_df["day_of_week"] = model_df.index.dayofweek
model_df["day_of_month"] = model_df.index.day
model_df["month"] = model_df.index.month
model_df["is_weekend"] = (model_df.index.dayofweek >= 5).astype(int)
```

```
# Past consumption values (at forecast origin t)
for lag in [1, 2, 3, 24, 48, 168]:
    model_df[f"lag_{lag}h"] = model_df["target_power_kw"].shift(lag)

model_df["rolling_mean_3h"] = model_df["target_power_kw"].shift(1).rolling(3).mean()
model_df["rolling_mean_24h"] = model_df["target_power_kw"].shift(1).rolling(24).mean()
model_df["rolling_std_24h"] = model_df["target_power_kw"].shift(1).rolling(24).std()

# One-hour-ahead target: consumption at t+1.
model_df["y_next_hour"] = model_df["target_power_kw"].shift(-1)

feature_cols = [
    "hour", "day_of_week", "day_of_month", "month", "is_weekend",
    "lag_1h", "lag_2h", "lag_3h", "lag_24h", "lag_48h", "lag_168h",
    "rolling_mean_3h", "rolling_mean_24h", "rolling_std_24h"
]
model_df = model_df.replace([np.inf, -np.inf], np.nan)
model_df = model_df.dropna(subset=feature_cols + ["y_next_hour"])

X = model_df[feature_cols]
y = model_df["y_next_hour"]
print("Model matrix:", X.shape)
print("Forecast target: mean active power in the next hour (kW)")
display(model_df[feature_cols + ["y_next_hour"]].head())
```

```
Model matrix: (33999, 14)
Forecast target: mean active power in the next hour (kW)
```

datetime	hour	day_of_week	day_of_month	month	is_weekend	lag_1h	lag_2h	lag_3h	lag_24h	lag_48h	lag_168h	rolli
2006-12-23 17:00:00	17	5	23	12	1	4.349100	4.049100	3.757967	1.496800	1.752633	4.222889	
2006-12-23 18:00:00	18	5	23	12	1	5.452533	4.349100	4.049100	2.686967	2.443300	3.632200	
2006-12-23 19:00:00	19	5	23	12	1	3.879400	5.452533	4.349100	3.938167	2.197133	3.400233	
2006-12-23 20:00:00	20	5	23	12	1	4.117833	3.879400	5.452533	3.536067	2.437367	3.268567	
2006-12-23 21:00:00	21	5	23	12	1	4.181400	4.117833	3.879400	4.548667	0.982267	3.056467	

7. Chronological train / validation / test split

```
# Never shuffle time-series rows: train on the past, test on the future.
n = len(model_df)
train_end = int(n * 0.70)
val_end = int(n * 0.85)

X_train, y_train = X.iloc[:train_end], y.iloc[:train_end]
X_val, y_val = X.iloc[train_end:val_end], y.iloc[train_end:val_end]
X_test, y_test = X.iloc[val_end:], y.iloc[val_end:]

print(f"Train: {len(X_train):,} rows | {X_train.index.min()} to {X_train.index.max()}")
print(f"Validation: {len(X_val):,} rows | {X_val.index.min()} to {X_val.index.max()}")
print(f"Test: {len(X_test):,} rows | {X_test.index.min()} to {X_test.index.max()}")
```

```
Train: 23,799 rows | 2006-12-23 17:00:00 to 2009-09-15 17:00:00
Validation: 5,100 rows | 2009-09-15 18:00:00 to 2010-04-19 18:00:00
Test: 5,100 rows | 2010-04-19 19:00:00 to 2010-11-26 20:00:00
```

8. Baseline and model training

```
models = {
    "Baseline (training mean)": DummyRegressor(strategy="mean"),
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest": RandomForestRegressor(
```

```
        n_estimators=150, min_samples_leaf=2, max_features=0.9,
        n_jobs=-1, random_state=RANDOM_STATE
    ),
    "Extra Trees": ExtraTreesRegressor(
        n_estimators=150, min_samples_leaf=2, max_features=0.9,
        n_jobs=-1, random_state=RANDOM_STATE
    ),
    "Hist Gradient Boosting": HistGradientBoostingRegressor(
        max_iter=150, learning_rate=0.08, l2_regularization=0.1,
        random_state=RANDOM_STATE
    )
}

def regression_metrics(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred) ** 0.5
    # MAPE is unstable when actual values are near zero; report only over non-near-zero actuals.
    yt = np.asarray(y_true)
    yp = np.asarray(y_pred)
    mask = np.abs(yt) > 0.05
    mape = (np.mean(np.abs((yt[mask] - yp[mask]) / yt[mask])) * 100) if mask.any() else np.nan
    return {"MAE": mae, "RMSE": rmse, "MAPE_%_actual_gt_0.05": mape, "R2": r2_score(y_true, y_pred)}

validation_rows = []
fitted_models = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    fitted_models[name] = model
    pred = model.predict(X_val)
    validation_rows.append({"Model": name, **regression_metrics(y_val, pred)})

validation_results = pd.DataFrame(validation_rows).sort_values("RMSE")
display(validation_results.style.format(precision=4))
```

	Model	MAE	RMSE	MAPE_%_actual_gt_0.05	R2
5	Hist Gradient Boosting	0.4460	0.6181	52.1165	0.5194
3	Random Forest	0.4441	0.6242	51.9365	0.5099
4	Extra Trees	0.4467	0.6243	52.9586	0.5097
2	Ridge Regression	0.5652	0.7507	72.4061	0.2910
1	Linear Regression	0.5652	0.7507	72.4063	0.2910
0	Baseline (training mean)	0.7324	0.9077	94.9199	-0.0365

9. Select model and evaluate once on the held-out test set

```
# Choose the model with the lowest validation RMSE.
best_name = validation_results.iloc[0]["Model"]
best_model = fitted_models[best_name]
test_pred = best_model.predict(X_test)

test_results = pd.DataFrame([{"Model": best_name, **regression_metrics(y_test, test_pred)}])
print("Selected by validation RMSE:", best_name)
display(test_results.style.format(precision=4))
```

```
Selected by validation RMSE: Hist Gradient Boosting
```

	Model	MAE	RMSE	MAPE_%_actual_gt_0.05	R2
0	Hist Gradient Boosting	0.3958	0.5455	57.1879	0.4088

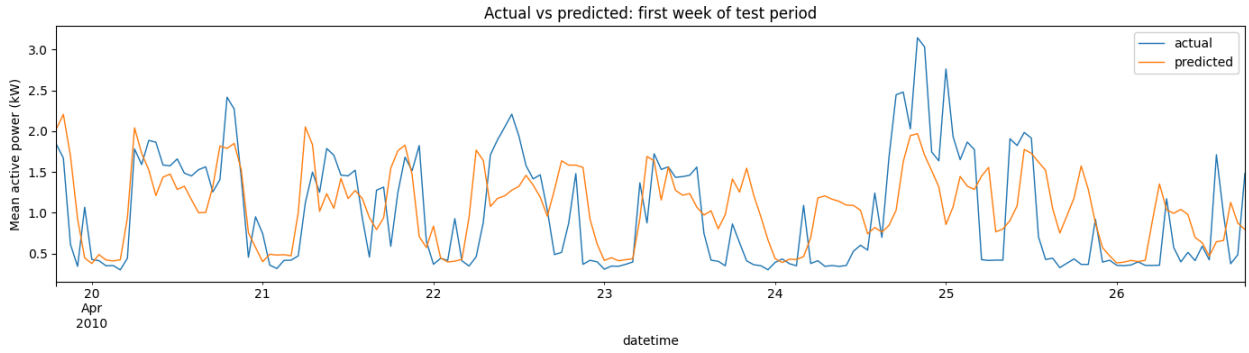
10. Results visualisation and residual analysis

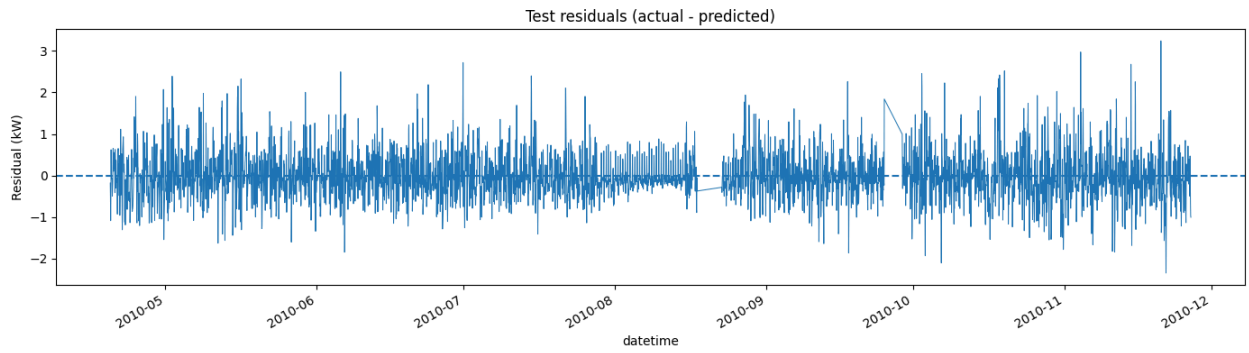
```
results = pd.DataFrame({
    "actual": y_test,
    "predicted": test_pred
}, index=y_test.index)
results["residual"] = results["actual"] - results["predicted"]

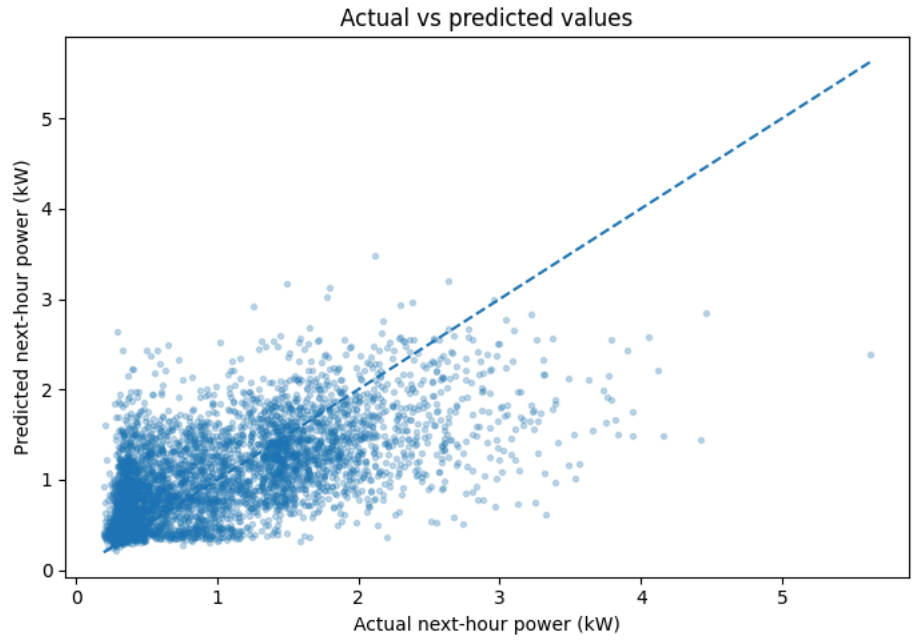
fig, axes = plt.subplots(2, 1, figsize=(14, 8))
results.iloc[:24*7][["actual", "predicted"]].plot(ax=axes[0], linewidth=1)
axes[0].set_title("Actual vs predicted: first week of test period")
axes[0].set_ylabel("Mean active power (kW)")
results["residual"].plot(ax=axes[1], linewidth=0.7, title="Test residuals (actual - predicted)")
```

```
axes[1].axhline(0, linestyle="--")
axes[1].set_ylabel("Residual (kW)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 5))
plt.scatter(results["actual"], results["predicted"], alpha=0.25, s=8)
lims = [
    min(results["actual"].min(), results["predicted"].min()),
    max(results["actual"].max(), results["predicted"].max())
]
plt.plot(lims, lims, linestyle="--")
plt.xlabel("Actual next-hour power (kW)")
plt.ylabel("Predicted next-hour power (kW)")
plt.title("Actual vs predicted values")
plt.tight_layout()
plt.show()
```







11. Feature importance / interpretation

```
# Permutation importance on a chronological validation sample.
# Sample for speed while preserving the time ordering.
```

```
sample_size = min(5000, len(X_val))
X_imp = X_val.iloc[:sample_size]
y_imp = y_val.iloc[:sample_size]

try:
    perm = permutation_importance(
        best_model, X_imp, y_imp,
        scoring="neg_root_mean_squared_error",
        n_repeats=3, random_state=RANDOM_STATE, n_jobs=-1
    )
    importance = pd.DataFrame({
        "feature": feature_cols,
        "importance_mean": perm.importances_mean,
        "importance_std": perm.importances_std
    }).sort_values("importance_mean", ascending=False)
    display(importance)
    importance.sort_values("importance_mean").plot(
        x="feature", y="importance_mean", kind="barh", legend=False, figsize=(9, 6),
        title="Permutation importance (validation sample)"
    )
    plt.xlabel("Increase in RMSE when feature is permuted (higher = more important)")
    plt.tight_layout()
    plt.show()
except Exception as e:
    print("Permutation importance could not be computed:", repr(e))
```

Next steps: Generate code with importance | New interactive sheet

	feature	importance_mean	importance_std
0	hour	0.250971	0.012712
5	lag_1h	0.149968	0.003147
1	day_of_week	0.038340	0.002477
6	lag_2h	0.003577	0.000385
2	day_of_month	0.002887	0.000643
11	rolling_mean_3h	0.001284	0.000203
9	lag_48h	0.000962	0.000201
4	is_weekend	0.000000	0.000000

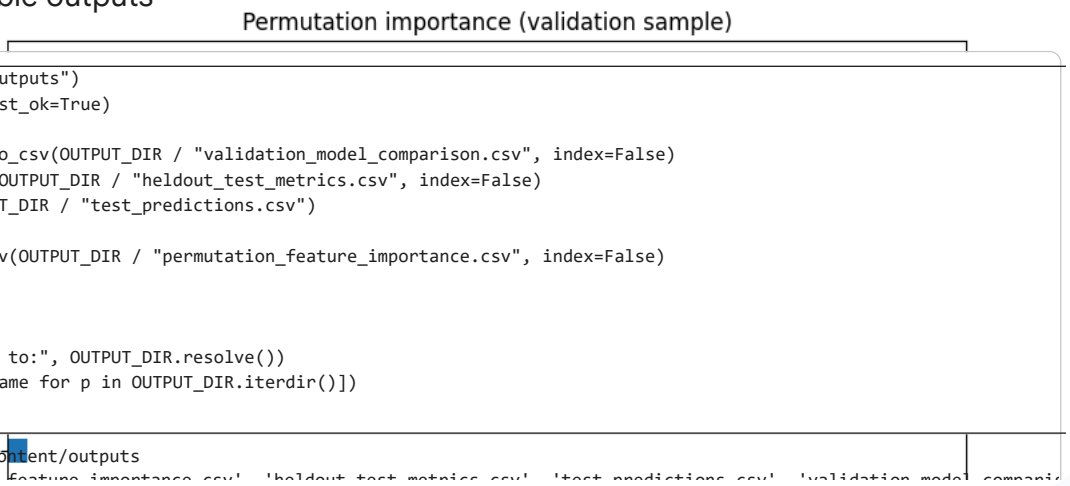
12. Predict the next hour from the latest available history

```
# Use the final feature row (whose lag/rolling features only use known historical readings).
latest_X = X.iloc[[-1]]
latest_origin = latest_X.index[0]
next_hour_estimate = float(best_model.predict(latest_X)[0])
print("Latest feature timestamp:", latest_origin)
print("Forecast target hour:", latest_origin + pd.Timedelta(hours=1))
print(f"Predicted mean active power for next hour: {next_hour_estimate:.3f} kW")
print("\nNote: This is an offline demonstration using the last timestamp in the historical dataset, not a live household fc
```

```
Latest feature timestamp: 2010-11-26 20:00:00
Forecast target hour: 2010-11-26 21:00:00
Predicted mean active power for next hour: 1.939 kW

Note: This is an offline demonstration using the last timestamp in the historical dataset, not a live household forecast.
```

13. Save reproducible outputs




```
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

validation_results.to_csv(OUTPUT_DIR / "validation_model_comparison.csv", index=False)
test_results.to_csv(OUTPUT_DIR / "heldout_test_metrics.csv", index=False)
results.to_csv(OUTPUT_DIR / "test_predictions.csv")
try:
    importance.to_csv(OUTPUT_DIR / "permutation_feature_importance.csv", index=False)
except NameError:
    pass

print("Saved results to:", OUTPUT_DIR.resolve())
print("Files:", [p.name for p in OUTPUT_DIR.iterdir()])
```

```
Saved results to: /content/outputs
Files: ['permutation_feature_importance.csv', 'heldout_test_metrics.csv', 'test_predictions.csv', 'validation_model_comparis
```